

Лабораторная работа №7 – «Модули ядра»

Введение

Модуль ядра – это некий код, который может быть загружен или выгружен ядром по мере необходимости. Модули расширяют функциональные возможности ядра без необходимости перезагрузки системы. Например, одна из разновидностей модулей ядра – драйверы устройств, которые позволяют ядру взаимодействовать с аппаратурой компьютера. При отсутствии поддержки модулей нам пришлось бы писать монолитные ядра и добавлять новые возможности прямо в ядро. При этом после добавления в ядро новых возможностей пришлось бы перезагружать систему.

Пример 1. hello-1.c

```
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("A simple Hello World kernel module");

int init_module(void)
{
    printk("<1>Hello world 1.\n");
    return 0;
}

void cleanup_module(void)
{
    printk(KERN_ALERT "Goodbye world 1.\n");
}
```

- `<linux/module.h>` - содержит определение макросов и функций для модулей (например, `MODULE_LICENSE`);
- `<linux/kernel.h>` - содержит определение уровня логирования `KERN_ALERT` и функцию `printk`;
- `init_module` – функция инициализации (вызывается командой `insmod`, когда загружается модуль);
- `printk` - это функция для печати сообщений из ядра (kernel print);
- `cleanup_module` – функция завершения (вызывается командой `rmmod`, когда выгружается модуль);
- `KERN_ALERT` – макрос уровня приоритета (лог-уровня) в ядре, который используется в функции `printk` (равен `<1>`);

Пример 2. Makefile для модуля ядра

```
obj-m += hello-1.o
```

Команда для сборки:

```
make -C /usr/src/linux-headers-$(uname -r) M=$PWD modules
```

- `make -C` – ключ `-C` заставляет `make` временно перейти в указанную директорию, где лежат заголовочные файлы и сборочные скрипты текущего ядра (модули нельзя собирать отдельно от исходников ядра);
- `M=$PWD` – возвращаемся в текущую папку (где лежит код) и собираем модуль именно в этой папке;
- `modules` – цель сборки (нужно собрать именно внешние модули);

Команда для загрузки модуля:

```
sudo insmod ./hello-1.ko
```

Команда для выгрузки модуля:

```
sudo rmmod hello-1
```

Пример 3. hello-2.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Demonstration of module_init and module_exit macros");

static int __init hello_2_init(void)
{
    printk(KERN_ALERT "Hello world 2\n");
    return 0;
}

static void __exit hello_2_exit(void)
{
    printk(KERN_ALERT "Goodbye, world 2\n");
}

module_init(hello_2_init);
module_exit(hello_2_exit);
```

- `<linux/init.h>` – содержит макросы `__init`, `__exit`, `module_init`, `module_exit`;

- `__init` – вынуждает ядро после выполнения инициализации модуля освободить память, занимаемую функцией (это относится только к встроенным модулям);
- `__exit` – вынуждает ядро освободить память, занимаемую функцией (это относится только к встроенным модулям);
- `module_init()`, `module_exit()` – можно называть функции инициализации и завершения как угодно, а не строго `init_module` и `cleanup_module`;

Пример 4. Makefile для сборки обоих модулей

```
obj-m += hello-1.o
obj-m += hello-2.o
```

Пример 5. hello-3.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Demonstration of __init, __initdata and __exit macros");

static int hello3_data __initdata = 3;

static int __init hello_3_init(void)
{
    printk(KERN_ALERT "Hello, world %d\n", hello3_data);
    return 0;
}

static void __exit hello_3_exit(void)
{
    printk(KERN_ALERT "Goodbye, world 3\n");
}

module_init(hello_3_init);
module_exit(hello_3_exit);
```

- `__initdata` – работает аналогично `__init`, но для переменных;

Пример 6. hello-4.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

#define DRIVER_AUTHOR "Peter Jay Salzman <p@dirac.org>"
#define DRIVER_DESC "A sample driver"
```

```

static int __init init_hello_4(void)
{
    printk(KERN_ALERT "Hello, world 4\n");
    return 0;
}

static void __exit cleanup_hello_4(void)
{
    printk(KERN_ALERT "Goodbye, world 4\n");
}

module_init(init_hello_4);
module_exit(cleanup_hello_4);

MODULE_LICENSE("GPL");
MODULE_AUTHOR(DRIVER_AUTHOR);
MODULE_DESCRIPTION(DRIVER_DESC);
MODULE_SUPPORTED_DEVICE("testdevice");

```

- MODULE_LICENSE() – макрос, задающий условия лицензирования модуля;
- MODULE_DESCRIPTION() – макрос для описания модуля;
- MODULE_AUTHOR() – макрос для установления авторства;
- MODULE_SUPPORTED_DEVICE() – макрос для описания типов устройств, поддерживаемых модулем;

Пример 7. hello-5.c

```

#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/stat.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Peter Jay Salzman");

static short int myshort = 1;
static int myint = 420;
static long int mylong = 9999;
static char *mystring = "blah"

module_param(myshort, short, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP);
MODULE_PARM_DESC(myshort, "A short integer");

module_param(myint, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
MODULE_PARM_DESC(myshort, "An integer");

module_param(mylong, long, S_IRUSR);
MODULE_PARM_DESC(mylong, "A long integer");

```

```

module_param(mystring, charp, 0000);
MODULE_PARM_DESC(mystring, "A character string");

static int __init hello_5_init(void)
{
    printk(KERN_ALERT "Hello, world 5\n=====\n");
    printk(KERN_ALERT "myshort is a short integer: %hd\n", myshort);
    printk(KERN_ALERT "myint is an integer: %d\n", myint);
    printk(KERN_ALERT "mylong is a long integer: %ld\n", mylong);
    printk(KERN_ALERT "mystring is a string: %s\n", mystring);
    return 0;
}

static void __exit hello_5_exit(void)
{
    printk(KERN_ALERT "Goodbye, world 5\n");
}

module_init(hello_5_init);
module_exit(hello_5_exit);

```

- MODULE_PARM_DESC() – макрос для описания входных аргументов модуля;
- USR (User) – владелец;
- GRP (Group) – группа;
- OTH (Others) – все остальные;
- R (Read) – чтение;
- W (Write) – запись;
- X (Execute) – выполнение;

Загрузка модуля:

```
sudo insmod hello-5.ko mystring="bebop" myshort=255
```

```
sudo insmod hello-5.ko mystring="supercall..." myint=100
```

```
sudo insmod hello-5.ko mylong=hello // Ошибка!
```

Пример 8. start.c

```

#include <linux/kernel.h>
#include <linux/module.h>

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Multi-file kernel module example (start part)");

int init_module(void)
{
    printk("Hello, world - this is the kernel speaking\n");
    return 0;
}

```

Пример 9. stop.c

```
#include <linux/kernel.h>
#include <linux/module.h>
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Multi-file kernel module example (stop part)");

void cleanup_module(void)
{
    printk("<1>Short is the life of a kernel module\n");
}
```

Пример 10. Makefile для сборки всех модулей

```
obj-m += hello-1.o
obj-m += hello-2.o
obj-m += hello-3.o
obj-m += hello-4.o
obj-m += hello-5.o
obj-m += startstop.o
startstop-objs := start.o stop.o
```

- Чтобы собрать модуль startstop.ko, нужно слинковать вместе объектные файлы start.o и stop.o